Lab 5 Report: Localization using Monte Carlo Algorithm

Team 11

Yuga Patel (6-4)
Avishai Jeselsohn (2) - Editor
Florida Cishesa (2)
Tyler Proctor (6-2)

6.420 Robotics Science and Systems

April 10, 2026

1 Introduction - Avishai

Localization, a crucial aspect for the success of an autonomous vehicle is the vehicle's ability to determine it's orientation and position in a known environment. Without a successful localization method autonomy fails, where a vehicle cannot navigate a world it cannot place itself in. As shown in Figure 1, localization allows to accurately predict a car's location with minimum error, allowing for the car to successfully maneuver through different paths and around obstacles. Within the context of RSS and this lab, the goal of localization is for the racecar to successfully determine its location within predetermined maps of the Stata Basement. This lab represents a critical junction between solving reactive problems in previous labs, such as line-following or wall-following, to intelligent problems of localization.

The problem lies within the inaccuracy of certain measurements from odometry data and LiDAR scans to accurately determine the robot's position with no uncertainty. To solve this, the car implemented a Monte Carlo Localization (MCL) algorithm, a probabilistic algorithm that represents the car's position as a distribution of thousands of "particles," or hypothesized states. As shown in Figure 2, MCL integrates two key components: a Motion Model, which uses odometry to predict particle movement, and a Sensor Model, which assigns weights to those particles by comparing live LiDAR scans against a pre-exisiting map. By recursively resampling the particle cloud based on these weights, the algorithm discards unlikely particles and converges on the car's true location.

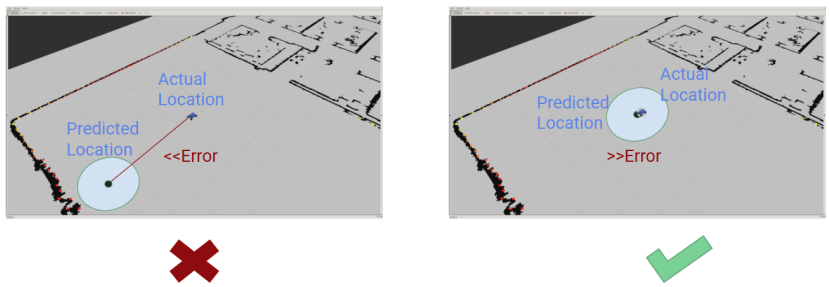


Figure 1: The goal is to minimize pose estimation error both in simulation and in the physical world, by combining motion-model and sensor-model

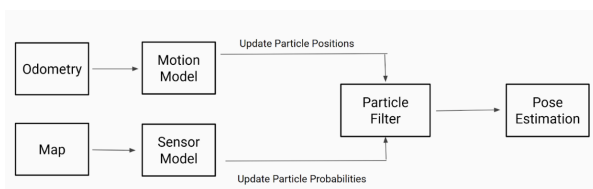


Figure 2: Logic flow diagram of a MCL algorithm implementation. Odometry data updates the motion model and the LiDAR scans based off the map updates the sensor model. These are then placed within a particle filter which outputs a estimated location. This process loops after every odometry and LiDAR scan callback.

2 Technical Approach

2.1 Motion Model - Florida

Motion Model is a module that particle filter utilizes to estimate the poses of the set of particles that represent potential poses of the car as illustrated in Figure 3. Given the pose array at time $k-1$ of the set of particles, this module returns a new estimated pose array at time k.

This module continuously reads the car's instantaneous linear and angular velocities from the odometry ($\dot{p}^o_{k-1}$) message, as well as computing the time increment between two subsequent readings Δt. These quantities were used to obtain incremental pose vector (Δp^o):

$$\dot{p}^o_{k-1} = \begin{pmatrix} \dot{x}^o_{k-1} \\ \dot{y}^o_{k-1} \\ \dot{\theta}^o_{k-1} \end{pmatrix} \tag{1}$$

$$\Delta p^o = \begin{pmatrix} \Delta x^o \\ \Delta y^o \\ \Delta \theta^o \end{pmatrix} = \dot{p}^o_{k-1}\,\Delta t \tag{2}$$

Given that that the odometry may accumulate errors over time in a real world environment, to avoid relying completely on the odometry data to compute the

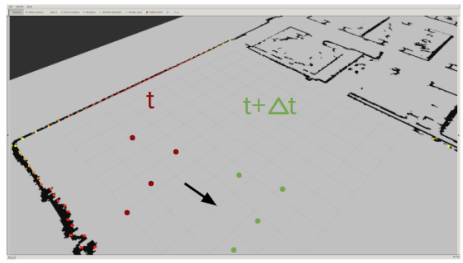


Figure 3: Given positions and angles (poses) of a set of particles at time t, motion model returns their new estimated poses at time t+Δt

incremental change, gaussian noise (Σ) is added to Δp^o

$$\Sigma = \begin{pmatrix} random(0, \sigma_x) \\ random(0, \sigma_y) \\ random(0, \sigma_\theta) \end{pmatrix} \tag{3}$$

where σ_x, σ_y, and σ_θ are tuning parameters for the sampling range of the gaussian noise value in their respective coordinates.

$$\Delta p^o_{noise} = \Delta p^o + \Sigma \tag{4}$$

The particle pose at time k (p^o_k) then estimates using the rotation matrix of the car at time k-1 ($R^0(\theta_{k-1})$) and Δp^o_{noise}:

$$R^0(\theta_{k-1}) = \begin{pmatrix} \cos(\theta_{k-1}) & -\sin(\theta_{k-1}) \\ \sin(\theta_{k-1}) & \cos(\theta_{k-1}) \end{pmatrix} \tag{5}$$

$$p^0_k \approx p^0_{k-1} + \begin{pmatrix} R^0(\theta_{k-1}) & 0 \\ 0^\top & 1 \end{pmatrix} \Delta p^0_{noise} \tag{6}$$

The model then publishes the estimated pose at time k p^0_k for all the particles, which the particle filter uses to estimate the racecar's potential position on the map.

2.2 Sensor Model - Yuga

Given the data from the LiDAR scan and a hypothesized pose from the motion model, the sensor model determines how likely a point is the actual pose. To enhance the run-time, a lookup table was pre-computed and defined as follows:

Pre-Computed table

For each scan that comes in, it is necessary to obtain a probability of the observed range z given a true range of d. Hence, it is defined a 201 x 201 table where the rows represent the range z values [0,200] and the columns represent

the true range d. Each of the entry represents $P(z|d)$, a weighted sum.

$$P(z|d) = \alpha_{hit} \cdot P_{hit}(z|d) + \alpha_{short} \cdot P_{short}(z|d) + \alpha_{max} \cdot P_{max}(z) + \alpha_{rand} \cdot P_{rand}(z)$$

P_{hit} is the Gaussian centered at d with standard deviation of 8, and the distribution defines the accurate measurement of estimation. P_{short} is where $z < d$, and the process measures the unexpected obstacles that are closer than the approximate value of d. P_{max} is when $z = z_{max}$ which usually happens due to sensor saturation. P_{rand} is just a uniform distribution of $1/z_{max}$ accounting for random noises. Weights were defined as follows from given weights in Part A of the lab: $\alpha_{hit} = 0.50, \alpha_{short} = 0.08, \alpha_{max} = 0.08, \alpha_{rand} = 0.34$ Here the probabilities add up to 1.

Runtime Evaluation

At runtime, evaluate() scores all N particles in parallel. Each particle's pose was ray-traced using RangeLibc's PyScanSimulator2D, producing an N × K matrix of expected ranges in meters. Both simulated and real ranges are converted to pixel indices by dividing by (map resolution × lidar-scale-to-map-scale), clipped to [0, 200], and cast to integers. It is then necessary to index into the precomputed table: for beam k of particle i, looking up in the sensor model table[z_k, d_{ik}] to get P(z_k — d_{ik}). To combine beam probabilities, it is necessary to sum log-probabilities rather than multiplying raw values, avoiding numerical underflow. The final weight for particle i is: $w_i = \exp(\sum_k \log P(z_k \mid d_{ik}))$
Ray tracing, unit conversion, clipping, table indexing, and log summation use NumPy vectorized operations, faster for real-time performance.

2.3 Particle Filter - Yuga

The particle filter implemented Monte Carlo Localization (MCL) loop which includes initialization, sensor update, resampling, and pose estimation. For the lab, N = 200 particles were found to be effective upon iteration.

Initialization

Particles were initialized using RViz's Pose Estimation button. When a pose-estimation is selected (x, y, θ) N particles are sampled from Gaussian centered at pose with $\sigma_{xy} = 0.5m$ and $\sigma_{theta} = 0.3$ rad. These numbers were selected upon iterations of testing and they work well to converge the pose estimation as the race-car moves.

Sensor Update and Resampling

The sensor updates with each new LiDAR scan and sample the scan down to K beams using spaced out indices. If indices are NaN or inf then they are made $range_{max}$. Then the sensor model returns the weighted vector which is raise

to a cubic root before normalizing. The cube root flattens the distribution to maintain diversity among particles while favoring the high-weighted particles. Then np.random.choice was to perform resampling with replacement, taking N particles proportional to their normal weights. Then small Gaussian noise $\sigma_{xy} = 0.01, \sigma_{theta} = 0.005$ was added to avoid sample depletion.

Pose Estimation

Simply averaging all particles doesn't work when they're spread across multiple clusters. To fix the problem, we compute a 2D histogram (20×20 bins) over particle positions, pick the bin with the most particles, and only average those. The pose is published as an Odometry message on /pf/pose/odom and as a TF transform from /map to the particle filter frame.

3 Experimental Evaluation

3.1 Simulation Localization Testing - Avishai

The goal of simulation testing was to ensure the motion model and sensor allowed for a successful convergence and approximation of the vehicle's position over time. Practically, this meant ensuring the particles remained packed around the vehicle, not spreading out without converging back. To create a better estimation, the noise parameters in the motion model, as explained, were tuned to obtain a better estimation. Within simulation, the true odometry data remained noiseless, creating an environment that does not represent the real world. As such, the parameters that were set for simulation did not translate perfectly to the real-world environment. however, the parameters were tuned to obtain an estimation deemed successful enough to then move towards testing on the vehicle itself. Two tests were conducted within simulation to allow for tuning of the noise parameters:

1. Straight-Line Test: driving at 1 m/s in a straight line.
2. Circle-Test: driving at 1 m/s in a circle.

3.1.1 Straight-Line Test

For the straight-line test, as shown in Figure 4(a), the vehicle drove straight down the simulated hallway at 1 m/s.[1]. The final parameters used for this simulation were: $\sigma_x = .2, \sigma_y = .1, \sigma_\theta = .15$. For the simulation tests, because the odometry data possessed no external or internal noise, an euclidean distance error analysis was conducted between the particle filter pose and real pose as shown in Figure 4(b), and found the average distance error to be 0.277 meters. Given the fact that the estimation will not be perfect due to the way the particle filter works, as well as the fact that the odometry data does not represent real

[1] Link to simulation video: https://drive.google.com/file/d/1AoNKNY0Y2oAOqARpxGCPtUTcKbQs05Sn/view?usp=sharing

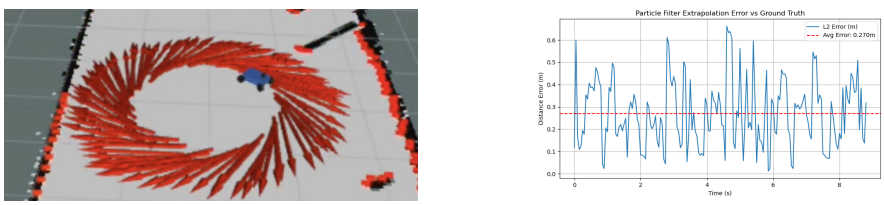
(a) (b)

Figure 4: (a) Photo from simulated straight line test showing the car driving with particle points in red surrounding the car with the estimated pose and orientation arrow in green. (b) Euclidean distance error analysis plot for the straight line test, showing the error distance from the real position over time, with an average line showing average distance error.

world odometry data, this error was deemed successful for straight-line driving. Given the success of these two tests, the car was then capable of moving towards testing in a real world environment, where noise plays a larger factor.

3.1.2 Circle Test

For the straight-line test, as shown in Figure 5(a), the vehicle drove in a circle in a simulated hallway at 1 m/s.[2].



(a) (b)

Figure 5: (a) Photo from simulated circle line test showing the car driving in a circle with current and prior estimated pose and orientation arrows in red. (b) Euclidean distance error analysis plot for the circle line test, showing the error distance from the real position over time, with an average line showing average distance error.

The final parameters used for this simulation were: $\sigma_x = .2, \sigma_y = .1, \sigma_\theta = .15$, the same as the straight-line test. The same euclidean distance error analysis was conducted between the particle filter pose and real pose as shown in Figure 5(b), and found the average distance error to be 0.277 meters. Similar to the reasoning for the straight-line test, this error was deemed successful for circle-line driving.

[2]Link to simulation video: https://drive.google.com/file/d/178shjJQblaZPUwDSfOncQEGOSWN-aCMw/view?usp=sharing

3.2 Real World Localization Testing - Tyler

When testing with the real car, the car should successfully localize, matching the particle filter position shown on RViz with the actual location of the car. This led to a decision to test on two different types of paths, similar to the paths given on simulation: a straight-line path down one of the hallways in the Stata basement and a circle path also in the basement. Additionally, the vehicle would intentionally swerve while driving, allowing to test for the accuracy of the orientation weight. For these tests, different levels of noise were inputted to highlight the successful performance of the particle filter. This led to implementation of three different noise level test cases for both the straight line and circle tests: high-level, medium-level, low-level.

3.2.1 Straight-Line Test

The racecar performed three different straight-line tests, using the long hallway in the Stata basement behind 32-082 as its testing environment. These tests focused on adjusting the noise level while having the robot perform the same route. The first test utilized the highest reasonable noise for the localization system, with $\sigma_x = .13, \sigma_y = .05, \sigma_\theta = .15$. This resulted in a noticeable spread of the particles, as seen in the following RViz screenshot of localization working off the real racecar.

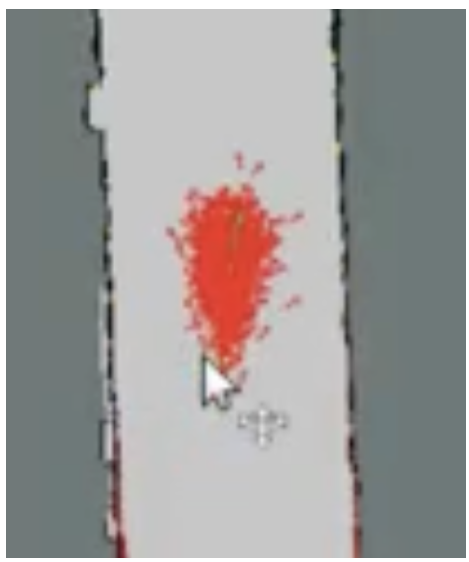
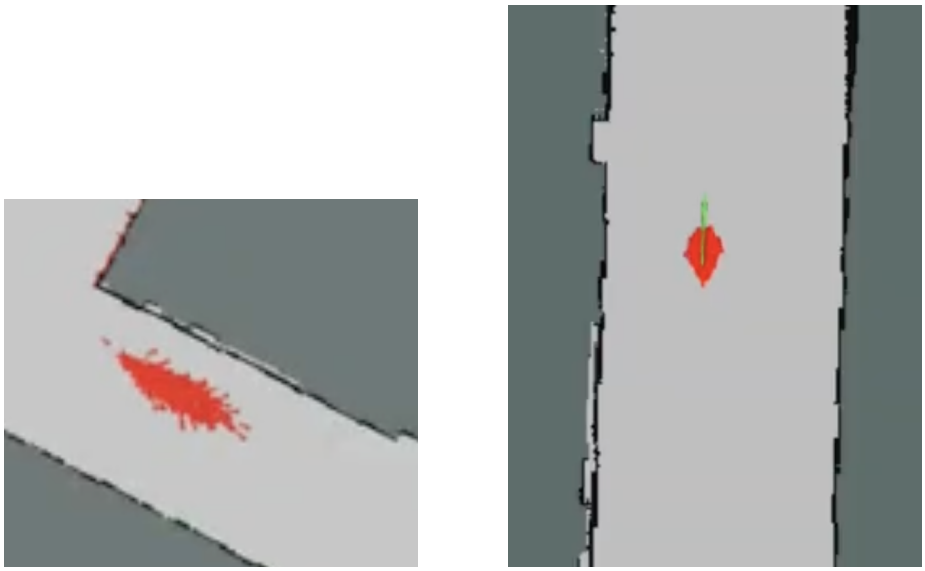


Figure 6: High Noise Test. Each red dot represents a particle within the particle filter. This screenshot represents the motion model updating the particle filter, with the particles starting to diverge prior to convergence due to the sensor model.

Tests were then done with a medium level of noise, $\sigma_x = .08, \sigma_y = .02, \sigma_\theta = .08$, and with a very low level of noise, $\sigma_x = .04, \sigma_y = .005, \sigma_\theta = .01$. A noticeable

difference between the tests was the size of the particle cloud surrounding the car in the particle filter. While each level of noise successfully followed the same path of the car, the size of spreading out of particles due to the updating motion model decreased as the noise level decreased. This was to be expected, however it is important to note that despite the differences in noise levels, each test qualitatively produced successful particle filters.



(a) (b)

Figure 7: (a) Divergence period during medium noise test, with red dots representing particles. Compared to the high noise test, the particle filter outputs a smaller area probability cloud. (b) low noise test showing a divergence period, where the sensor model updates the particle filter and causes the particles to converge to a higher density area.

3.2.2 Circle Test

The car performed three circle tests, using the taped orange circle seen in Stata Basement. Here, there was a similar focus with the tests on adjusting the noise level, and used the same three tiers of noise that were used for the straight-line test. Similarly for these tests, the noise level affected the level of spread of particles around the car as it performed localization while moving around the path. This led to very similar spreads to what was seen in Figures 6 and 7.
Localization was successfully established in both the straight-line test cases and the circle test cases. To see videos of the localization module working well as the robot is moving down these different routes, refer to slides 11-15 of the briefing presentation.[3]

[3]Link to Briefing slides: https://docs.google.com/presentation/d/1TF96mIfd3UTC1XoHricmaoBb3lgC8_-2LE863FCCP_NM/edit?usp=sharing

4 Conclusion - Avishai

The car successfully achieved localization through implementing the Monte-Carlo Localization algorithm. By effectively fusing odometry data with LiDAR scans, the vehicle can maintain a stable estimate of its pose within the pre-determined Stata map. While the qualitative results are promising, work remains to minimize the variance between the particle filter's estimated pose and the car's actual physical state. Given the inherent noiseness of the real odometry data, to quantitatively evaluate the error, pre-determined paths will have to be measured in the real world and the "ground truth" coordinates will have to be compared to the estimated coordinates. This benchmarking will allow us to fine-tune the sensor model's parameters and optimize the particle count for a better balance between accuracy and computational latency.
This localization provides the prerequisite for the upcoming path-planning lab. In the autonomous navigation pipeline, path planning is the process of generating a collision-free trajectory from point A to point B. However, a plan is only as good as the state estimate it relies on; if the localization is inaccurate, the car will attempt to follow a trajectory that does not align with the physical world, leading to collisions or failed maneuvers. Successful localization ensures that future path-planning algorithms have the necessary tools to navigate successfully though different environments.

5 Lessons Learned

Florida - In this lab, I learned the importance of good communication and teamwork. Knowing that I was most likely not to be productive over spring break, I planned before the break to get a head start on the lab so that my teammates would have material to start from. Also, The past weeks have been very busy for all of us, and some of us could not be available to come in at usual teamwork times. Nevertheless, we informed each other in advance and planned to make as much progress as we could before taking other commitments. This way of communication helped us complete the lab deliverables in a timely manner. On the technical side, I grew a more solid understanding of how to adapt from perfect conditions to real-life application. For instance, in the physical world, estimating the car's position simply from Odometry data may yield high error. This is because the car may slip, the floor may have unexpected textures, and there may be other unexpected factors as well. Adding noise to the perfect conditions combining two or more sensor readings, may help include these external factors and get closer to the true value.
Avishai - In this lab, I learned lessons on both the technical and communication sides. On the technical side, I learned the math involved with the motion and sensor model, and I realized that having a complete understand of the math allowed for a smooth transition to implementing them in code and later tuning the necessary parameters. I also learned the tradeoffs between accuracy and precision by tuning the parameters of the motion model and seeing its effect

in Rviz. On the communication side, I felt that we were a lot more successful this time around at communicating when we needed help, when a member was going to miss any time, and delegated tasks successfully. I learned that if we all understand our roles then we can work a lot more efficiently and don't just sit there not knowing what to do. I felt we really took the lessons and experiences from the prior labs and implemented them in this lab, and as a result we grew as a team.

Tyler - In this lab, I learned the importance of ensuring the system and code performs well on sim and covers as many edge cases as possible. We did a good job as a team of taking the time to understand how motion and sensor model worked, how particle filter combined the two, and then tinkering and improving on the sim as much as possible before ever putting the code on the robot. In turn however, this made the work of testing the code in the real world a lot easier. It took barely any tuning or anything to get the code to work. This made collecting results and having final data very easy to get done. As a team, I think we learned more about setting a realistic and rigid schedule for everyone to follow that made it easier for tasks to be done on time and progress to continually occur. This helped us produce our final end result.

Yuga - Technically, lab 5 exposed me to what localization is, how we as engineers can use it when developing different technologies in the future, and what components are necessary in ensuring that it works properly. I learned about the sensor model is a very important technical components that helps the car understand where it is compared to the objects around it, and I hope that the knowledge is useful to be applied in future engineering problems. Lab 5 was also the first time our team implemented strict deadlines that we had to follow, and the deadlines kept us on track. Our work over the break also made our lives easier when we just had testing left to do. I learned that we will be most successful when we do a lot fo work in the beginning.